



Tecnológico de Monterrey

Desarrollo de aplicaciones avanzadas de ciencias computacionales - TC3002B.503

RETO: Análisis Semántico

Santiago Gutiérrez González | A00572499

Alejandro Moncada Espinosa | A01638343

Ana Camila Jimenez Mendoza | A01174422

Jorge Iván Sánchez González | A01761414

Andres Benjamin Antelis Moreno | A01637683

Mini-Triton: Diseño de Gramática y Parser

1. Introducción

Este documento describe la gramática propuesta para Mini-Triton, un subconjunto controlado de Triton orientado a practicar diseño de gramática, parsing descendente y construcción de AST. La implementación desarrollada acepta un programa formado por exactamente un kernel decorado con '@triton.jit', con bloques delimitados por llaves, sentencias terminadas en punto y coma, y expresiones aritméticas con precedencia definida.

El parser fue implementado con un enfoque **recursive descent**, por lo que la gramática se organizó en niveles de precedencia y sin recursión izquierda directa.

2. Gramática en EBNF

None

```
program      = decorator, kernel ;

decorator    = "@", "triton", ".", "jit" ;

kernel       = "def", identifier, "(", params, ")", ":",
               "{", stmt_list, "}" ;

params       = [ param, { ",", param } ] ;

param        = identifier, [ ":", identifier, ".",
                             identifier ] ;

stmt_list    = { stmt } ;

stmt         = assign | expr_stmt ;
```

```

assign      = identifier, "=", expr, ";" ;

expr_stmt   = expr, ";" ;

expr        = term, { ("+" | "-"), term } ;

term        = factor, { ("*" | "/"), factor } ;

factor      = "(", expr, ")" | number | name_or_call ;

name_or_call = qualified_name, [ "(", args, ")" ] ;

qualified_name = identifier, [ ".", identifier ] ;

args        = [ expr, { ",", expr } ] ;

identifier  = token IDENTIFIER ;

number      = token INTEGER | token FLOAT ;

```

3. Explicación Breve de Decisiones

3.1 Un solo kernel por programa

La producción inicial es `program`, que obliga a reconocer primero el decorador y después exactamente un kernel. Además, el parser valida que no queden tokens extra al final del archivo. Con esto se garantiza que no se aceptan múltiples funciones o kernels en el mismo programa.

3.2 Decorador obligatorio

El kernel solo se considera válido si inicia con:

```
None
@triton.jit
```

Esto se modela como decorator para dejar claro que no es opcional.

3.3 Bloques con llaves

En Mini-Triton no se usa indentación estilo Python. Por esa razón, la producción `kernel` exige explícitamente:

```
None
":" "{" stmt_list "}"
```

De esta manera, el parser distingue sin ambigüedad el inicio y fin del bloque del kernel.

Eso quiere decir que para que nuestro programa sea válido debe iniciar y finalizar con brackets.

3.4 Distinción entre asignación y sentencia de expresión

Las sentencias permitidas son solo dos:

1. Asignación
2. Sentencia de expresión

Para diferenciarlas, el parser usa una estrategia de ver hacia adelante, el famoso LookAhead:

- Si el token actual es un `identifier` y el siguiente es `"="`, se parsea una asignación.
- En cualquier otro caso, se parsea una sentencia de expresión.

Esto evita ambigüedades entre un nombre simple y el lado izquierdo de una asignación.

3.5 Parámetros y anotaciones

La lista de parámetros acepta parámetros simples como:

```
None  
x
```

y parámetros anotados como:

```
None  
BS: tl.constexpr
```

En la implementación actual, la anotación se modela de forma general como ``identifier "." identifier``, aunque el caso esperado por la especificación es ``tl.constexpr``.

3.6 Llamadas y nombres calificados

Las llamadas soportadas tienen dos formas:

```
None  
id(args)  
  
tl.id(args)  
  
id.id(args)
```

Para reflejar esto, la gramática usa ``qualified_name``, que permite un identificador solo o un identificador con un segmento adicional separado por punto.

3.7 Argumentos posicionales

Los argumentos de una llamada se separan por comas y cada uno debe ser una expresión válida. No se aceptan argumentos nombrados como:

None

```
mask = 1
```

porque están fuera del alcance definido para Mini-Triton.

4. Precedencia y Asociatividad

La precedencia de operadores se resuelve separando la gramática de expresiones en varios no terminales:

- `factor`: paréntesis, números, nombres y llamadas
- `term`: multiplicación y división
- `expr`: suma y resta

Esto produce la siguiente jerarquía:

None

```
( ) > llamadas/variables/numeros > * y / > + y -
```

4.1 Asociatividad

Los operadores binarios se implementan como asociativos por la izquierda.

Por ejemplo:

None

```
a - b - c
```

se interpreta como:

```
None  
(a - b) - c
```

y:

```
None  
a / b / c
```

se interpreta como:

```
None  
(a / b) / c
```

4.2 Ejemplo de parseo

La expresión:

```
None  
a + b * c
```

se construye como:

```
None  
BinaryOp("+", Name(a), BinaryOp("*", Name(b), Name(c)))
```

lo cual demuestra que `\*` tiene mayor precedencia que `+`.

De forma similar, la expresión:

None

```
(a + b) * c
```

se interpreta primero resolviendo el paréntesis y luego aplicando la multiplicación.

5. Diseño del AST

El AST fue diseñado para representar la estructura sintáctica esencial del programa Mini-Triton. Los nodos principales son los siguientes:

5.1 Program

Representa el programa completo y contiene un único kernel.

None

```
Program {  
  
    kernel: AstNode  
  
}
```

5.2 Kernel

Representa la definición del kernel: nombre, parámetros y cuerpo.

None

```
Kernel {  
  
    name: String,  
  
    params: Vec<AstNode>,  
  
    body: Vec<AstNode>
```



```
}
```

5.3 Param

Representa un parámetro del kernel, con o sin anotación.

```
None  
Param {  
  
    name: String,  
  
    annotation: Option<String>  
  
}
```

5.4 Assign

Representa una asignación.

```
None  
Assign {  
  
    name: String,  
  
    value: AstNode  
  
}
```

5.5 ExprStmt

Representa una sentencia compuesta solo por una expresión, por ejemplo una llamada a función.

None

```
ExprStmt {  
  
    expr: AstNode  
  
}
```

5.6 BinaryOp

Representa una operación binaria aritmética.

None

```
BinaryOp {  
  
    op: String,  
  
    left: AstNode,  
  
    right: AstNode  
  
}
```

5.7 Call

Representa una llamada a función o primitiva.

None

```
Call {  
  
    callee: String,
```

```
args: Vec<AstNode>

}
```

5.8 Name

Representa una referencia al identificador.

```
None
Name {

    id: String

}
```

5.9 Number

Representa un literal numérico entero o flotante.

```
None
Number {

    value: String

}
```

6. Ejemplo de AST

Para el siguiente programa:

```
Python
@triton.jit

def k(x, out, BS: tl.constexpr): {

    pid = tl.program_id(0);

    offs = pid * BS + tl.arange(0, BS);

    tl.store(out + offs, tl.load(x + offs));

}
```

un AST en texto puede verse así:

```
None
Program(

  Kernel(name="k", params=[x, out, BS:tl.constexpr], body=[

    Assign(pid, Call("tl.program_id", [Number(0)])),

    Assign(offs, BinaryOp("+",

      BinaryOp("*", Name(pid), Name(BS)),

      Call("tl.arange", [Number(0), Name(BS)])

    )),

    ExprStmt(
```

```

    Call("tl.store", [
        BinaryOp("+", Name(out), Name(offsets)),
        Call("tl.load", [
            BinaryOp("+", Name(x), Name(offsets))
        ])
    ])
])
)
]
)

```

7. Conclusiones

La gramática propuesta cumple con los requisitos centrales de Mini-Triton:

- exactamente un kernel por archivo
- decorador obligatorio `@triton.jit`
- bloques con llaves
- sentencias terminadas en `;`
- asignaciones y sentencias de expresión
- expresiones con precedencia y asociatividad bien definidas
- llamadas con argumentos posicionales

- soporte de parámetros anotados

Además, el AST obtenido conserva la información necesaria para etapas posteriores como validación semántica, análisis adicional o generación de código.